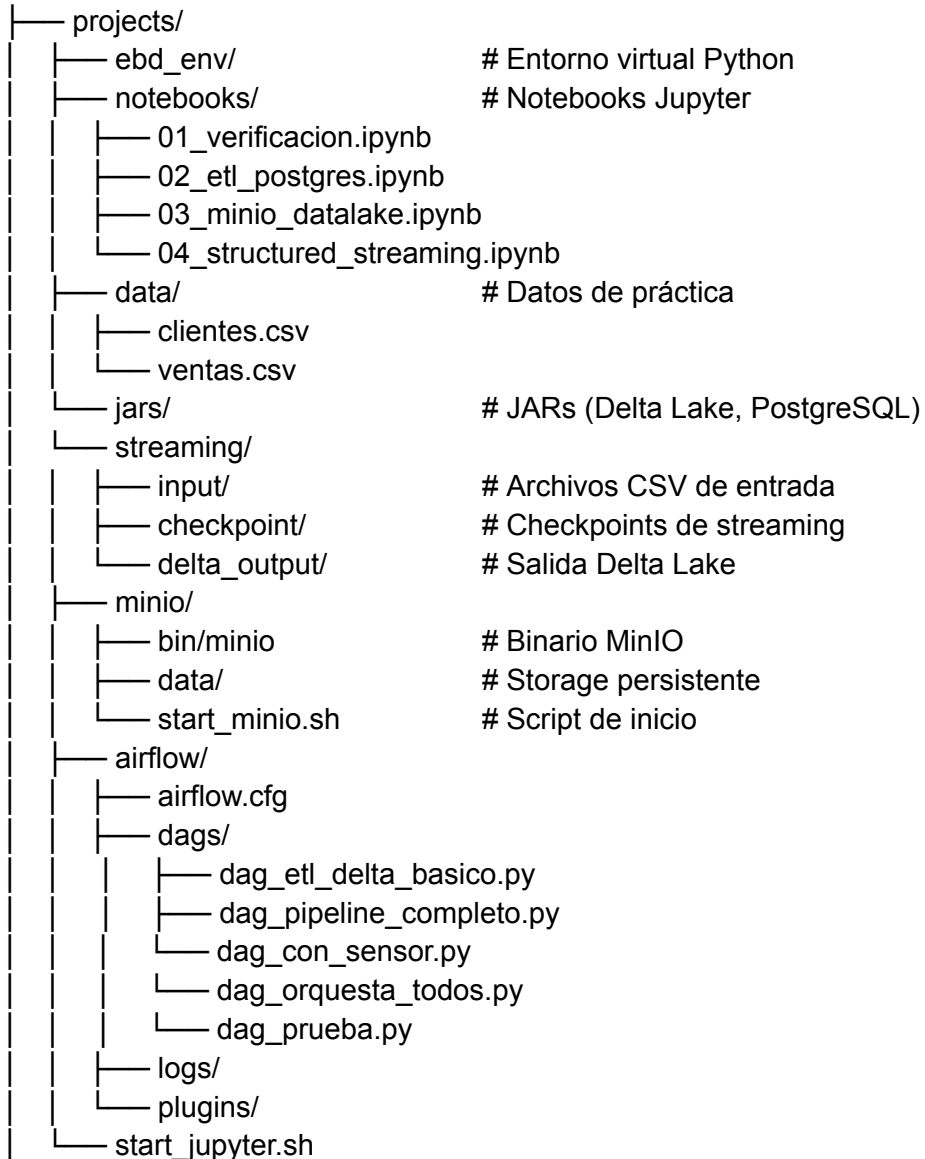


UNIVERSIDAD DE NARIÑO
Especialización en Construcción de Software

Guía 3: Procesamiento de Flujos y Orquestación de Pipelines

Estructura de Directorios

/home/<su_usuario>/



PARTE 1: STRUCTURED STREAMING

1. Preparar Directorios de Streaming

```
Shell
# Crear estructura de directorios para streaming
mkdir -p ~/projects/streaming/{input,checkpoint,delta_output}

# Crear archivo CSV inicial de prueba
cat > ~/projects/streaming/input/ventas_001.csv << 'EOF'
id_venta,id_cliente,producto,cantidad,precio_unitario,fecha
V001,C001,Laptop Pro,1,2500.00,2024-05-01
V002,C002,Mouse Gamer,2,45.50,2024-05-01
V003,C001,Teclado Mecánico,1,120.00,2024-05-02
EOF

echo "Directorios de streaming creados"
ls -la ~/projects/streaming/
```

Salida esperada:

```
Shell
drwxrwxr-x 5 test test 4096 Jun 18 22:38 .
drwxrwxr-x 8 test test 4096 Jun 18 22:37 ..
drwxrwxr-x 2 test test 4096 Jun 18 22:38 checkpoint
drwxrwxr-x 2 test test 4096 Jun 18 22:38 delta_output
drwxrwxr-x 2 test test 4096 Jun 18 22:42 input
```

Notebook - Structured Streaming (04_structured_streaming.ipynb)

Objetivo:

Ingestión continua de CSV → Delta Lake con checkpoints, triggers y tolerancia a fallos.

Asegúrese de estar en directorio projects y cargar Jupyter

```
Shell
# Entrar a la carpeta projects
cd ~/projects

# Inicie Notebook de Jupyter
./start_jupyter.sh
```

1. Celda 1: Verificar Estructura de Directorios

Python

```
import os

print("=" * 60)
print("VERIFICACIÓN DE DIRECTORIOS DE STREAMING")
print("=" * 60)

paths = {
    "Input": os.path.expanduser("~/projects/streaming/input"),
    "Checkpoint": os.path.expanduser("~/projects/streaming/checkpoint"),
    "Delta Output": os.path.expanduser("~/projects/streaming/delta_output")
}

for name, path in paths.items():
    exists = os.path.exists(path)
    status = "OK" if exists else "FAIL"
    print(f"    {status} {name}: {path}")

# Listar archivos iniciales
input_files = os.listdir(paths["Input"]) if os.path.exists(paths["Input"]) else []
print(f"\nArchivos en input/: {input_files}")
```

Salida esperada:

Shell

```
=====
VERIFICACIÓN DE DIRECTORIOS DE STREAMING
=====
OK Input: /home/test/projects/streaming/input
OK Checkpoint: /home/test/projects/streaming/checkpoint
OK Delta Output: /home/test/projects/streaming/delta_output

Archivos en input/: ['ventas_001.csv']
```

2. Celda 2: Crear SparkSession con Streaming

Python

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType,
DoubleType, DateType

print("=" * 60)
```

```

print("CREACIÓN DE SPARKSESSION PARA STREAMING")
print("=" * 60)

spark = (SparkSession.builder
        .appName("StructuredStreaming")
        .master("local[*]")
        .config("spark.driver.memory", "1g")
        .config("spark.executor.memory", "1500m")
        .config("spark.sql.streaming.checkpointLocation",
os.path.expanduser("~/projects/streaming/checkpoint"))
        .config("spark.sql.extensions",
"io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .getOrCreate())

print("SparkSession creada")
print(f" App: {spark.sparkContext.appName}")
print(f" Checkpoint por defecto: ~/projects/streaming/checkpoint")

```

Salida esperada:

```

Shell
=====
CREACIÓN DE SPARKSESSION PARA STREAMING
=====

SparkSession creada
App: StructuredStreaming
Checkpoint por defecto: ~/projects/streaming/checkpoint

```

3. Celda 3: Definir Esquema y Configurar File Stream

```

Python
print("=" * 60)
print("CONFIGURACIÓN DEL FILE STREAM")
print("=" * 60)

# Esquema del archivo CSV de ventas
schema = StructType([
    StructField("id_venta", StringType(), False),
    StructField("id_cliente", StringType(), False),

```

```

    StructField("producto", StringType(), False),
    StructField("cantidad", IntegerType(), False),
    StructField("precio_unitario", DoubleType(), False),
    StructField("fecha", DateType(), True)
])

input_path = os.path.expanduser("~/projects/streaming/input")
checkpoint_path = os.path.expanduser("~/projects/streaming/checkpoint")
output_path = os.path.expanduser("~/projects/streaming/delta_output")

print(f"Input: {input_path}")
print(f"Checkpoint: {checkpoint_path}")
print(f"Output: {output_path}")

# Configurar lectura de stream
df_stream = (spark.readStream
              .schema(schema)
              .option("header", "true")
              .option("maxFilesPerTrigger", 1) # Procesar 1 archivo por trigger
              .csv(input_path))

print("Stream configurado")
print("Esquema:")
df_stream.printSchema()

```

Salida esperada:

```

Shell
=====
CONFIGURACIÓN DEL FILE STREAM
=====
Input: /home/test/projects/streaming/input
Checkpoint: /home/test/projects/streaming/checkpoint
Output: /home/test/projects/streaming/delta_output
Stream configurado
Esquema:
root
 |-- id_venta: string (nullable = true)
 |-- id_cliente: string (nullable = true)
 |-- producto: string (nullable = true)
 |-- cantidad: integer (nullable = true)
 |-- precio_unitario: double (nullable = true)
 |-- fecha: date (nullable = true)

```

4. Celda 4: Escribir Stream a Delta Lake

Python

```
from pyspark.sql.functions import col, current_timestamp, lit

print("=" * 60)
print("ESCRITURA DEL STREAM A DELTA LAKE")
print("=" * 60)

# Agregar metadatos de procesamiento
df_enriched = df_stream.withColumn("fecha_procesamiento", current_timestamp()) \
    .withColumn("batch_id", lit("batch_streaming"))

# Configurar escritura
query = (df_enriched.writeStream
    .format("delta")
    .outputMode("append")
    .option("checkpointLocation", checkpoint_path)
    .trigger(processingTime="10 seconds") # Trigger cada 10 segundos
    .start(output_path))

print("Stream iniciado")
print(f"Query name: {query.name}")
print(f"Status: {query.status}")
print(f"ID: {query.id}")

# Esperar unos segundos para procesar archivos iniciales
import time
time.sleep(5)
```

Salida esperada:

Shell

```
=====
ESCRITURA DEL STREAM A DELTA LAKE
=====

Stream iniciado
Query name: None
Status: {'message': 'Initializing sources', 'isDataAvailable': False, 'isTriggerActive': False}
ID: 983498a4-11b3-4d43-b165-e5e435100e23
```

5. Celda 5: Verificar Datos Procesados

Python

```
print("=" * 60)
print("VERIFICACIÓN DE DATOS PROCESADOS")
print("=" * 60)

# Leer datos procesados
df_delta = spark.read.format("delta").load(output_path)

print(f"Registros procesados: {df_delta.count()}")
print("\nDatos:")
df_delta.show(truncate=False)

print("\nEstadísticas:")
df_delta.groupBy("fecha").count().orderBy("fecha").show()
```

Salida esperada:

Shell

Registros procesados: 3

Datos:

id_venta	id_cliente	producto	cantidad	precio_unitario	fecha	fecha_procesamiento	batch_id
V001	C001	Laptop Pro	1	2500.0	2024-05-01	2026-06-18 23:01:49.596	batch_streaming
V002	C002	Mouse Gamer	2	45.5	2024-05-01	2026-06-18 23:01:49.596	batch_streaming
V003	C003	Teclado Mecanico	1	120.0	2024-05-02	2026-06-18 23:01:49.596	batch_streaming

Estadísticas:

fecha	count
2024-05-01	2
2024-05-02	1

6. Celda 6: Simular Nuevos Datos (Micro-batch)

Python

```
import time

print("=" * 60)
print("SIMULACIÓN DE NUEVOS DATOS")
print("=" * 60)

# Crear nuevo archivo CSV en input/
```

```

nuevas_ventas = """id_venta,id_cliente,producto,cantidad,precio_unitario,fecha
V004,C003,Monitor 4K,2,450.00,2024-05-03
V005,C002,Webcam HD,1,80.00,2024-05-03
V006,C004,Audífonos BT,3,60.00,2024-05-04"""

nuevo_archivo = os.path.expanduser("~/projects/streaming/input/ventas_002.csv")
with open(nuevo_archivo, 'w') as f:
    f.write(nuevas_ventas)

print(f"Archivo creado: {nuevo_archivo}")
print("Esperando procesamiento (15 segundos)...")
time.sleep(15)

# Verificar datos actualizados
df_delta_actualizado = spark.read.format("delta").load(output_path)
print(f"\nRegistros totales después del micro-batch:
{df_delta_actualizado.count()}")
df_delta_actualizado.orderBy("fecha").show(truncate=False)

```

Salida esperada:

Shell

```

=====
SIMULACIÓN DE NUEVOS DATOS
=====
Archivo creado: /home/test/projects/streaming/input/ventas_002.csv
Esperando procesamiento (15 segundos)...

Registros totales después del micro-batch:6
+-----+-----+-----+-----+-----+-----+-----+-----+
|id_venta|id_cliente|producto      |cantidad|precio_unitario|fecha      |fecha_procesamiento |batch_id  |
+-----+-----+-----+-----+-----+-----+-----+-----+
|V001    |C001      |Laptop Pro    |1        |2500.0         |2024-05-01|2026-06-18 23:01:49.596|batch_streaming|
|V002    |C002      |Mouse Gamer   |2        |45.5           |2024-05-01|2026-06-18 23:01:49.596|batch_streaming|
|V003    |C003      |Teclado Mecanico|1        |120.0          |2024-05-02|2026-06-18 23:01:49.596|batch_streaming|
|V004    |C003      |Monitor 4K    |2        |450.0          |2024-05-03|2026-06-18 23:12:30.096|batch_streaming|
|V005    |C002      |Webcam HD     |1        |80.0           |2024-05-03|2026-06-18 23:12:30.096|batch_streaming|
|V006    |C004      |Audífonos BT  |3        |60.0           |2024-05-04|2026-06-18 23:12:30.096|batch_streaming|
+-----+-----+-----+-----+-----+-----+-----+-----+

```

7. Celda 7: Verificar Checkpoint y Recuperación

Python

```

print("=" * 60)
print("VERIFICACIÓN DE CHECKPOINTS")
print("=" * 60)

```

```

# Listar contenido del directorio checkpoint
import subprocess
result = subprocess.run(["ls", "-la", checkpoint_path], capture_output=True,
text=True)
print("Contenido de checkpoint/:")
print(result.stdout)

# Detener y reiniciar el stream (simulación de fallo y recuperación)
print("\nDeteniendo stream...")
query.stop()

print("Reiniciando stream desde checkpoint...")
query2 = (df_enriched.writeStream
          .format("delta")
          .outputMode("append")
          .option("checkpointLocation", checkpoint_path)
          .trigger(processingTime="10 seconds")
          .start(output_path))

time.sleep(5)

# Verificar que no hay duplicados (exactly-once semantics)
df_final = spark.read.format("delta").load(output_path)
print(f"\nRegistros finales (sin duplicados): {df_final.count()}")
print("Recuperación desde checkpoint exitosa - Exactly-once garantizado")

query2.stop()

```

Salida esperada:

```

Shell
=====
VERIFICACIÓN DE CHECKPOINTS
=====
Contenido de checkpoint/:
total 28
drwxrwxr-x 5 test test 4096 Jun 18 23:01 .
drwxrwxr-x 5 test test 4096 Jun 18 22:38 ..
drwxr-xr-x 2 test test 4096 Jun 18 23:12 commits
-rw-r--r-- 1 test test  45 Jun 18 23:01 metadata
-rw-r--r-- 1 test test  12 Jun 18 23:01 .metadata.crc
drwxr-xr-x 2 test test 4096 Jun 18 23:12 offsets
drwxr-xr-x 3 test test 4096 Jun 18 23:01 sources

```

Deteniendo stream...

Reiniciando stream desde checkpoint...

26/06/18 23:20:09 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.

Registros finales (sin duplicados): 6

Recuperación desde checkpoint exitosa - Exactly-once garantizado

PARTE 2: INSTALACIÓN DE APACHE AIRFLOW

Definir Variables de Entorno

```
Shell
# Agregar a ~/.bashrc para persistencia
cat >> ~/.bashrc << 'EOF'

# === CONFIGURACIÓN APACHE AIRFLOW ===
export AIRFLOW_HOME=/home/<su_usuario>/airflow
export
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN=postgresql+psycopg2://postgres:123abc@localhost:5432/airflow_meta
export AIRFLOW__CORE__EXECUTOR=LocalExecutor
EOF

source ~/.bashrc

# Verificar
echo $AIRFLOW_HOME
# Salida: /home/<su_usuario>/airflow
```

Instalación Airflow (Modo Standalone)

```
Shell
# Asegurar entorno virtual activado
source ~/projects/ebd_env/bin/activate

# Instalar Airflow 3.2.2 con soporte PostgreSQL
pip install apache-airflow[postgres]==3.2.2

# Verificar instalación
airflow version
# Salida esperada: 3.2.2
```

Verificar Dependencias

```
Shell
# Verificar que psycopg2 está instalado (para PostgreSQL)
pip list | grep -E "apache-airflow|psycopg2|sqlalchemy"
```

Salida esperada:

```
Shell
apache-airflow 3.2.2
```

apache-airflow-core	3.2.2
apache-airflow-providers-common-compat	1.15.0
apache-airflow-providers-common-io	1.7.3
apache-airflow-providers-common-sql	2.0.1
apache-airflow-providers-postgres	6.7.1
apache-airflow-providers-smtp	3.0.1
apache-airflow-providers-standard	1.14.0
apache-airflow-task-sdk	1.2.2
psycpg2-binary	2.9.12

Crear Base de Datos para metadata de airflow

Shell

```
# Conectar como usuario postgres
sudo -u postgres psql
```

Dentro del prompt postgres=#:

SQL

```
-- Crear base de datos para metadata de Airflow
CREATE DATABASE airflow_meta;

-- Verificar que se creó
\l

-- Salir
\q
```

Verificar Conectividad

Shell

```
# Probar conexión desde terminal
psql -h localhost -p 5432 -U postgres -d airflow_meta -c "SELECT version();"

version
```

```
# Ingresar password: 123abc
# Debe mostrar la versión de PostgreSQL
```

```
-----
-----
PostgreSQL 16.14 (Ubuntu 16.14-0ubuntu0.24.04.1) on x86_64-pc-linux-gnu, compiled by
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, 64-bit
```

Crear Estructura de DAGs

```
Shell

mkdir -p ~/airflow/dags
mkdir -p ~/airflow/logs
mkdir -p ~/airflow/plugins

# Verificar estructura
ls -la ~/airflow/
```

Ejecutar migraciones

```
Shell

# Aplicar migraciones a la base de datos
airflow db migrate
```

Salida esperada:

```
Shell

2026-06-19T05:14:38.212094Z [info] ] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-06-19T05:14:38.212280Z [info] ] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-06-19T05:14:38.212364Z [info] ] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-06-19T05:14:38.212428Z [info] ] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-06-19T05:14:38.212485Z [info] ] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-06-19T05:14:38.212544Z [info] ] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
2026-06-19T05:14:38.876979Z [info] ] Performing upgrade to the metadata database [airflow.cli.commands.db_command] loc=db_command.py:134
url=postgresql+psycopg2://postgres:***@localhost:5432/airflow_meta
2026-06-19T05:14:38.883814Z [info] ] Context impl PostgresqlImpl. [alembic.runtime.migration] loc=migration.py:210
2026-06-19T05:14:38.885139Z [info] ] Will assume transactional DDL. [alembic.runtime.migration] loc=migration.py:213
2026-06-19T05:14:39.209797Z [info] ] Context impl PostgresqlImpl. [alembic.runtime.migration] loc=migration.py:210
2026-06-19T05:14:39.209978Z [info] ] Will assume transactional DDL. [alembic.runtime.migration] loc=migration.py:213
2026-06-19T05:14:39.210841Z [info] ] Creating Airflow database tables from the ORM [airflow.utils.db] loc=db.py:739
2026-06-19T05:14:39.210963Z [info] ] Creating global lock context [airflow.utils.db] loc=db.py:720
2026-06-19T05:14:39.234298Z [info] ] Pool status: Pool size: 5 Connections in pool: 0 Current Overflow: -3 Current Checked out connections: 2
[airflow.utils.db] loc=db.py:723
2026-06-19T05:14:39.234516Z [info] ] Creating metadata [airflow.utils.db] loc=db.py:725
2026-06-19T05:14:39.677295Z [info] ] Getting alembic config [airflow.utils.db] loc=db.py:728
2026-06-19T05:14:39.678045Z [info] ] Stamping migration head [airflow.utils.db] loc=db.py:731
2026-06-19T05:14:39.680929Z [info] ] Context impl PostgresqlImpl. [alembic.runtime.migration] loc=migration.py:210
2026-06-19T05:14:39.681119Z [info] ] Will assume transactional DDL. [alembic.runtime.migration] loc=migration.py:213
2026-06-19T05:14:39.767131Z [info] ] Running stamp_revision -> 1d6611b6ab7c [alembic.runtime.migration] loc=migration.py:621
2026-06-19T05:14:39.772553Z [info] ] Airflow database tables created [airflow.utils.db] loc=db.py:734
2026-06-19T05:14:39.838393Z [info] ] Database migration done! [airflow.cli.commands.db_command] loc=db_command.py:152
```

Verificar base de datos

```
Shell

airflow db check

# Salida esperada: Connection successful
```

```
(ebd_env) test@test:~$ airflow db check
2026-06-19T05:16:15.790948Z [info] ] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-06-19T05:16:15.791323Z [info] ] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-06-19T05:16:15.791428Z [info] ] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-06-19T05:16:15.791495Z [info] ] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-06-19T05:16:15.791555Z [info] ] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-06-19T05:16:15.791621Z [info] ] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
2026-06-19T05:16:16.330366Z [info] ] Connection successful. [airflow.utils.db] loc=db.py:1521
```

Opción A: Iniciar Airflow (Modo Standalone)

```
Shell  
airflow standalone
```

Salida esperada:

```
Shell  
Standalone | Starting Airflow Standalone  
Simple auth manager | Password for user 'admin': 377ce8f7R4nGxmHu  
standalone | Checking database is initialized
```

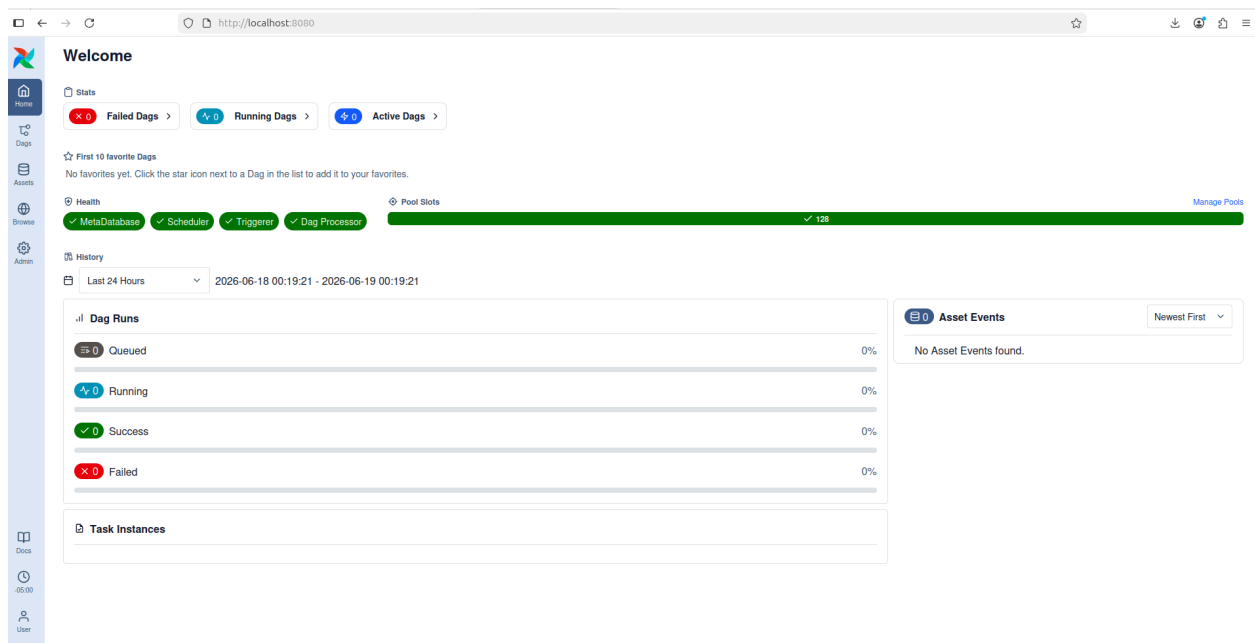
IMPORTANTE: Copiar la contraseña generada (las X). Es la única credencial de acceso.

Acceder a la Interfaz Web

URL: <http://localhost:8080>

Username: admin

Password: [la contraseña que mostró standalone]



Opción B: Iniciar Airflow (Segundo plano)

```
Shell  
# Iniciar en background  
nohup airflow standalone > ~/airflow/standalone.log 2>&1 &
```

```
# Verificar proceso
ps aux | grep airflow

# Ver logs
tail -f ~/airflow/standalone.log
```

Encontrar y Matar el Proceso

```
Shell

# 1. Buscar el PID (ID de proceso) de airflow standalone
pgrep -f "airflow standalone"
# 0
ps aux | grep "airflow standalone"

# 2. Matar el proceso (reemplaza <PID> por el número que muestra el comando anterior)
kill <PID>

# 3. Matar todos los procesos asociados a airflow
sudo pkill -f "airlfow"
```

Buscar la Contraseña en el Log

```
Shell

# La contraseña se guarda en el log que redirigiste
grep -i "password" ~/airflow/standalone.log

# 0 buscar más específicamente:
grep "Login with username" ~/airflow/standalone.log

# Ver las últimas 100 líneas del log
tail -n 100 ~/airflow/standalone.log | grep -A 2 -B 2 "password\\|Login"
```

PARTE 3: Desarrollo de DAGs

Notebook - 02_etl_delta_intro.ipynb

Objetivo:

Crear DataFrame, transformar, escribir/leer Delta Lake, demostrar MERGE (UPSERT) y Time Travel.

Asegúrese de estar en directorio projects y cargar Jupyter

Shell

```
# Entrar a la carpeta projects
cd ~/projects

# Inicie Notebook de Jupyter
./start_jupyter.sh
```

1. Celda 1: Crear SparkSession con Delta Lake

Python

```
from pyspark.sql import SparkSession

spark = (SparkSession.builder
        .appName("ETL_Delta_Intro")
        .master("local[*]")
        .config("spark.driver.memory", "1g")
        .config("spark.executor.memory", "1500m")
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog",
        "org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .getOrCreate())

print("SparkSession con Delta Lake creada")
print(f"Versión Spark: {spark.version}")
```

2. Celda 2: Crear Datos de Prueba (CSV simulado)

Python

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType

data = [
    ("P001", "Laptop Pro", "Electrónica", 2500.00, 10),
    ("P002", "Mouse Gamer", "Electrónica", 45.50, 50),
    ("P003", "Teclado Mecánico", "Electrónica", 120.00, 30),
```

```

        ("P004", "Monitor 4K", "Electrónica", 450.00, 15),
        ("P005", "Silla Ergonómica", "Muebles", 300.00, 8),
    ]

    schema = StructType([
        StructField("id_producto", StringType(), False),
        StructField("nombre", StringType(), False),
        StructField("categoria", StringType(), False),
        StructField("precio", DoubleType(), False),
        StructField("stock", IntegerType(), False),
    ])

    df = spark.createDataFrame(data, schema)
    print("DataFrame creado:")
    df.show()

```

3. Celda 3: Transformaciones Básicas

```

Python
from pyspark.sql.functions import col, when, lit, round as spark_round

df_transformado = (df
    .withColumn("precio_iva", spark_round(col("precio") * 1.19, 2))
    .withColumn("estado_stock",
        when(col("stock") > 20, "Suficiente")
        .when(col("stock") > 10, "Moderado")
        .otherwise("Bajo"))
    .withColumn("fecha_carga", lit("2024-05-20")))

print("Transformaciones aplicadas:")
df_transformado.show()

```

4. Celda 4: Escribir a Delta Lake

```

Python
import os

delta_path = os.path.expanduser("~/projects/data/productos_delta")

# Eliminar si existe (para pruebas limpias)
import shutil
if os.path.exists(delta_path):
    shutil.rmtree(delta_path)

```

```
df_transformado.write.format("delta").save(delta_path)
print(f"Delta Lake escrito en: {delta_path}")
```

5. Celda 5: Leer desde Delta Lake

Python

```
df_delta = spark.read.format("delta").load(delta_path)
print("Datos leídos desde Delta Lake:")
df_delta.show()
```

6. Celda 6: MERGE (UPSERT) – Actualizar e Insertar

Python

```
from delta import DeltaTable

# Nuevos datos: P001 actualiza precio, P006 es nuevo
nuevos_datos = [
    ("P001", "Laptop Pro", "Electrónica", 2700.00, 12), # UPDATE
    ("P006", "Webcam HD", "Electrónica", 80.00, 25),    # INSERT
]

df_nuevos = spark.createDataFrame(nuevos_datos, schema)

delta_table = DeltaTable.forPath(spark, delta_path)

# MERGE con columnas base
delta_table.alias("target").merge(
    df_nuevos.alias("source"),
    "target.id_producto = source.id_producto"
).whenMatchedUpdate(set={
    "nombre": "source.nombre",
    "categoria": "source.categoria",
    "precio": "source.precio",
    "stock": "source.stock"
}).whenNotMatchedInsert(values={
    "id_producto": "source.id_producto",
    "nombre": "source.nombre",
    "categoria": "source.categoria",
    "precio": "source.precio",
    "stock": "source.stock"
}).execute()
```

```

print("MERGE base ejecutado")

# RECALCULAR columnas derivadas en TODOS los registros
df_actualizado = spark.read.format("delta").load(delta_path)

df_recalculado = df_actualizado \
    .withColumn("precio_iva", spark_round(col("precio") * 1.19, 2)) \
    .withColumn("estado_stock",
        when(col("stock") > 20, "Suficiente")
        .when(col("stock") > 10, "Moderado")
        .otherwise("Bajo"))

# Sobrescribir la tabla Delta con valores recalculados
df_recalculado.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(delta_path)

print("Columnas derivadas recalculadas")
df_recalculado.show()

```

7. Celda 7: Verificar Resultado del MERGE

Python

```

df_resultado = spark.read.format("delta").load(delta_path)
print("Datos después del MERGE:")
df_resultado.orderBy("id_producto").show()

```

8. Celda 8: Time Travel – Ver Historial

Python

```

delta_table = DeltaTable.forPath(spark, delta_path)

print("Historial de versiones:")
delta_table.history().select("version", "timestamp",
"operation").show(truncate=False)

```

9. Celda 9: Time Travel – Leer Versión Anterior

Python

```

df_version_0 = spark.read.format("delta").option("versionAsOf",
0).load(delta_path)
print("Versión 0 (original, sin P006, P001=2500):")
df_version_0.select("id_producto", "nombre", "precio").show()

```

```
df_version_1 = spark.read.format("delta").option("versionAsOf",  
1).load(delta_path)  
print("Versión 1 (después de MERGE, con P006, P001=2700):")  
df_version_1.select("id_producto", "nombre", "precio").show()
```

DAG 1: dag_etl_delta_basico.py

Python

```
# ~/airflow/dags/dag_etl_delta_basico.py

from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta
import pendulum

local_tz = pendulum.timezone("America/Bogota")

default_args = {
    'owner': 'oscar',
    'depends_on_past': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

with DAG(
    dag_id='dag_etl_delta_basico',
    default_args=default_args,
    description='Ejecuta notebook 02 de ETL Delta Lake',
    schedule='@daily',
    start_date=datetime(2026, 6, 19, tzinfo=local_tz),
    catchup=False,
    tags=['etl', 'delta', 'basico'],
) as dag:

    ejecutar_notebook = BashOperator(
        task_id='ejecutar_notebook_02',
        bash_command="""
            source /home/su_usuario/projects/ebd_env/bin/activate && \
            cd /home/su_usuario/projects/notebooks && \
            jupyter nbconvert --to notebook --execute 02_etl_delta_intro.ipynb \
            --output 02_etl_delta_intro_executed.ipynb
        """,
    )

    verificar_salida = BashOperator(
        task_id='verificar_delta_output',
        bash_command="""
            ls -la /home/su_usuario/projects/data/*.delta 2>/dev/null || echo
            "Verificar path de salida"
        """,
    )
```

```
ejecutar_notebook >> verificar_salida
```

DAG 2: dag_pipeline_completo.py

Python

```
# ~/airflow/dags/dag_pipeline_completo.py

from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta
import pendulum
import os
import shutil

local_tz = pendulum.timezone("America/Bogota")

default_args = {
    'owner': 'oscar',
    'depends_on_past': False,
    'retries': 2,
    'retry_delay': timedelta(minutes=3),
}

def limpiar_directorios(**kwargs):
    dirs_to_clean = [
        os.path.expanduser("~/projects/streaming/checkpoint"),
        "/tmp/spark_temp"
    ]
    for d in dirs_to_clean:
        if os.path.exists(d):
            shutil.rmtree(d)
        os.makedirs(d, exist_ok=True)
    print("Directorios limpiados")

with DAG(
    dag_id='dag_pipeline_completo',
    default_args=default_args,
    description='Pipeline L-E-T-C con dependencias',
    schedule='0 2 * * *',
    start_date=datetime(2026, 6, 19, tzinfo=local_tz),
    catchup=False,
```

```

tags=['pipeline', 'completo', 'letc'],
) as dag:

limpiar = PythonOperator(
    task_id='limpiar_directorios',
    python_callable=limpiar_directorios,
)

extraer = BashOperator(
    task_id='extraer_delta',
    bash_command="""
        source /home/su_usuario/projects/ebd_env/bin/activate && \
        cd /home/su_usuario/projects/notebooks && \
        jupyter nbconvert --to notebook --execute 02_etl_delta_intro.ipynb \
        --output 02_executed.ipynb
    """,
)

transformar = BashOperator(
    task_id='transformar_postgres',
    bash_command="""
        source /home/su_usuario/projects/ebd_env/bin/activate && \
        cd /home/su_usuario/projects/notebooks && \
        jupyter nbconvert --to notebook --execute 03_etl_postgres.ipynb \
        --output 03_executed.ipynb
    """,
)

cargar = BashOperator(
    task_id='cargar_minio',
    bash_command="""
        source /home/su_usuario/projects/ebd_env/bin/activate && \
        cd /home/su_usuario/projects/notebooks && \
        jupyter nbconvert --to notebook --execute 04_minio_data_lake.ipynb \
        --output 04_executed.ipynb
    """,
)

limpiar >> extraer >> transformar >> cargar

```

DAG 3: dag_con_sensor.py

Python

```
# ~/airflow/dags/dag_con_sensor.py

from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.sensors.filesystem import FileSensor
from datetime import datetime, timedelta
import pendulum
import os

local_tz = pendulum.timezone("America/Bogota")

default_args = {
    'owner': 'oscar',
    'depends_on_past': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=1),
}

INPUT_DIR = os.path.expanduser("~/projects/streaming/input")

with DAG(
    dag_id='dag_con_sensor',
    default_args=default_args,
    description='Espera archivo y ejecuta ETL de streaming',
    schedule='*/10 * * * *',
    start_date=datetime(2026, 6, 19, tzinfo=local_tz),
    catchup=False,
    tags=['sensor', 'streaming', 'trigger'],
) as dag:

    esperar_archivo = FileSensor(
        task_id='esperar_archivo_entrada',
        filepath=os.path.join(INPUT_DIR, "nuevos_datos*.csv"),
        fs_conn_id='fs_default',
        poke_interval=30,
        timeout=600,
        mode='poke',
    )

    ejecutar_streaming = BashOperator(
        task_id='ejecutar_streaming',
        bash_command="""
            source /home/su_usuario/projects/ebd_env/bin/activate && \
            cd /home/su_usuario/projects/notebooks && \
        """
    )
```

```

        jupyter nbconvert --to notebook --execute
05_structured_streaming.ipynb \
        --output 05_executed.ipynb
        """
    )

    archivar = BashOperator(
        task_id='archivar_archivo',
        bash_command=f"""
            mkdir -p {INPUT_DIR}/procesados && \
            mv {INPUT_DIR}/nuevos_datos_*.csv {INPUT_DIR}/procesados/ 2>/dev/null
        || true
        """
    )

    esperar_archivo >> ejecutar_streaming >> archivar

```

DAG 4: dag_orquesta_todos.py

Python

```

# ~/airflow/dags/dag_orquesta_todos.py

from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta
import pendulum

local_tz = pendulum.timezone("America/Bogota")

default_args = {
    'owner': 'oscar',
    'depends_on_past': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=2),
}

def verificar_postgres(**kwargs):
    import psycopg2
    try:
        conn = psycopg2.connect(
            host="localhost",
            database="datamart",

```

```

        user="postgres",
        password="123abc"
    )
    cur = conn.cursor()
    cur.execute("SELECT version();")
    version = cur.fetchone()
    print(f"PostgreSQL conectado: {version[0]}")
    cur.close()
    conn.close()
    return True
except Exception as e:
    print(f"Error PostgreSQL: {e}")
    raise

def verificar_minio(**kwargs):
    import urllib.request
    try:
        response =
urllib.request.urlopen("http://localhost:9000/minio/health/live", timeout=5)
        if response.status == 200:
            print("MinIO está saludable")
            return True
    except Exception as e:
        print(f"MinIO no disponible: {e}")
        raise

with DAG(
    dag_id='dag_orquesta_todos',
    default_args=default_args,
    description='Orquesta notebooks 02+03+04 con verificaciones',
    schedule='0 */6 * * *',
    start_date=datetime(2026, 6, 19, tzinfo=local_tz),
    catchup=False,
    tags=['orquestacion', 'multi-tarea', 'completo'],
) as dag:

    check_postgres = PythonOperator(
        task_id='verificar_postgres',
        python_callable=verificar_postgres,
    )

    check_minio = PythonOperator(
        task_id='verificar_minio',
        python_callable=verificar_minio,
    )

```

```

run_02 = BashOperator(
    task_id='ejecutar_notebook_02',
    bash_command="""
        source /home/su_usuario/projects/ebd_env/bin/activate && \
        cd /home/su_usuario/projects/notebooks && \
        jupyter nbconvert --to notebook --execute 02_etl_delta_intro.ipynb \
        --output 02_orquestado.ipynb
    """,
)

run_03 = BashOperator(
    task_id='ejecutar_notebook_03',
    bash_command="""
        source /home/su_usuario/projects/ebd_env/bin/activate && \
        cd /home/su_usuario/projects/notebooks && \
        jupyter nbconvert --to notebook --execute 03_etl_postgres.ipynb \
        --output 03_orquestado.ipynb
    """,
)

run_04 = BashOperator(
    task_id='ejecutar_notebook_04',
    bash_command="""
        source /home/su_usuario/projects/ebd_env/bin/activate && \
        cd /home/su_usuario/projects/notebooks && \
        jupyter nbconvert --to notebook --execute 04_minio_data_lake.ipynb \
        --output 04_orquestado.ipynb
    """,
)

notificar = BashOperator(
    task_id='notificar_completado',
    bash_command='echo "Pipeline completado exitosamente a las $(date)"',
)

[check_postgres, check_minio] >> run_02 >> run_03 >> run_04 >> notificar

```

DAG 5: dag_prueba.py

Python

```
# ~/airflow/dags/dag_prueba.py
```

```

from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta
import pendulum

# Zona horaria local (ajusta según tu región)
local_tz = pendulum.timezone("America/Bogota")

default_args = {
    'owner': 'oscar',
    'depends_on_past': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=1),
}

with DAG(
    dag_id='dag_prueba_funcionamiento',
    default_args=default_args,
    description='DAG de prueba para verificar Airflow 3.2.2',
    schedule=None, # 'None' en lugar de 'None' (sin comillas) o '@once'
    start_date=datetime(2026, 6, 19, tzinfo=local_tz), # Fecha fija en lugar de
days_ago
    catchup=False,
    tags=['prueba', 'verificacion'],
) as dag:

    tarea_hola = BashOperator(
        task_id='saludar',
        bash_command='echo "Airflow 3.2.2 funcionando correctamente en VM
Ubuntu"',
    )

    tarea_fecha = BashOperator(
        task_id='mostrar_fecha',
        bash_command='date',
    )

    tarea_hola >> tarea_fecha

```

GUÍA: CÓMO EJECUTAR DAGs EN APACHE AIRFLOW 3.2.2

Pre-requisitos Verificados

Componente	Estado
Airflow 3.2.2 corriendo (airflow standalone)	OK
UI accesible en http://localhost:8080	OK
Usuario admin logueado	OK
DAGs en ~/airflow/dags/	OK
Notebooks en ~/spark_projects/notebooks/	OK

PASO 1: VERIFICAR QUE LOS DAGs ESTÁN EN AIRFLOW

1. Verificar Archivos en el Directorio Correcto

Shell

```
# En una nueva terminal  
ls -la ~/airflow/dags/
```

Salida esperada:

Shell

```
dag_etl_delta_basico.py  
dag_pipeline_completo.py  
dag_con_sensor.py  
dag_orquesta_todos.py  
dag_prueba_funcionamiento.py
```

PASO 2: ACCEDER A LA UI Y LOCALIZAR LOS DAGs

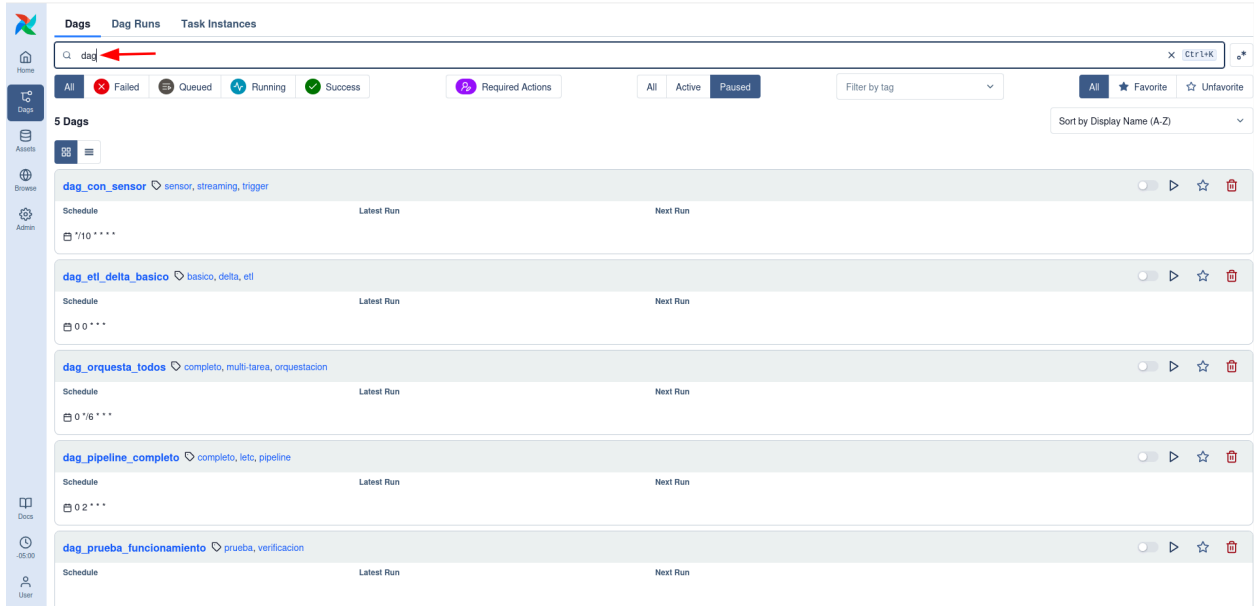
1. Abrir el navegador

Shell

```
URL: http://localhost:8080  
Usuario: admin  
Password: [la que generó standalone]
```

2. Navegar a la lista de DAGs

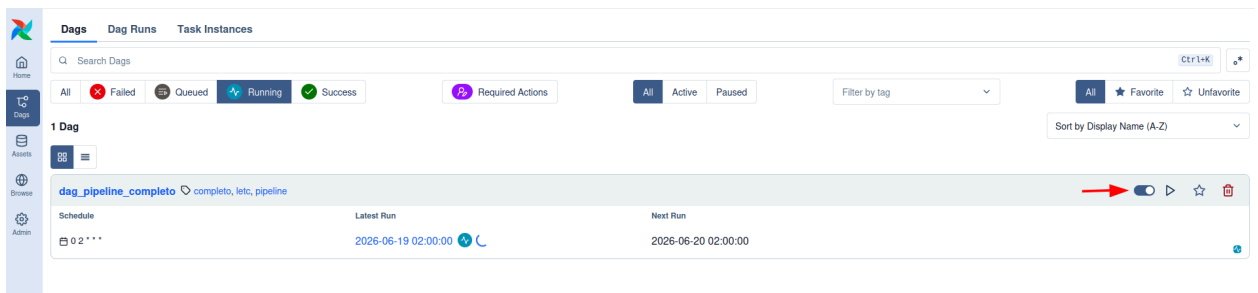
Clic en "DAGs" en el menú superior (o ve directamente a <http://localhost:8080/dags>)



PASO 3: EJECUTAR UN DAG (MODO MANUAL - TRIGGER)

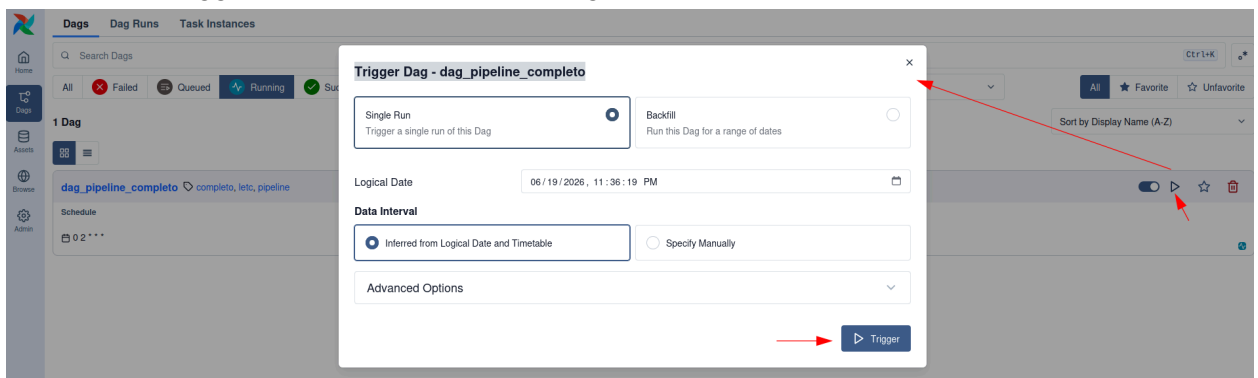
1. Activar el DAG

Clic en el toggle a la derecha del DAG dag_prueba_funcionamiento
El toggle debe cambiar a ON (azul)



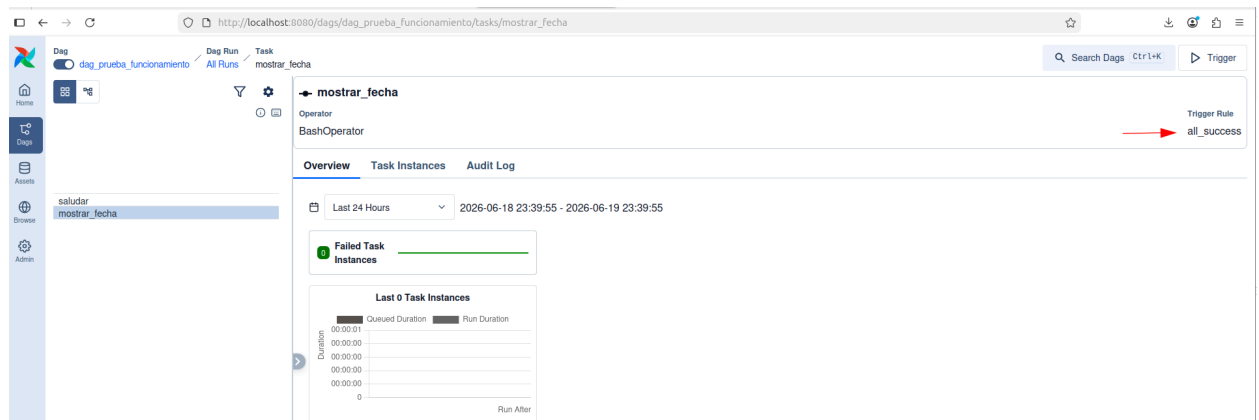
2. Ejecutar Manualmente (Trigger)

Haz clic en el botón (play) a la derecha del DAG
Selecciona "Trigger DAG" en el menú desplegable



3. Verificar Ejecución

Entrar al DAG haciendo clic en su nombre. Verás:



PASO 4: EJECUTAR EL DAG dag_etl_delta_basico

Este DAG ejecuta el Notebook 02 (02_etl_delta_intro.ipynb).

1. Verificar que el Notebook Existe

Shell

```
ls -la ~/spark_projects/notebooks/02_etl_delta_intro.ipynb
# Debe existir
```

2. Activar y Ejecutar

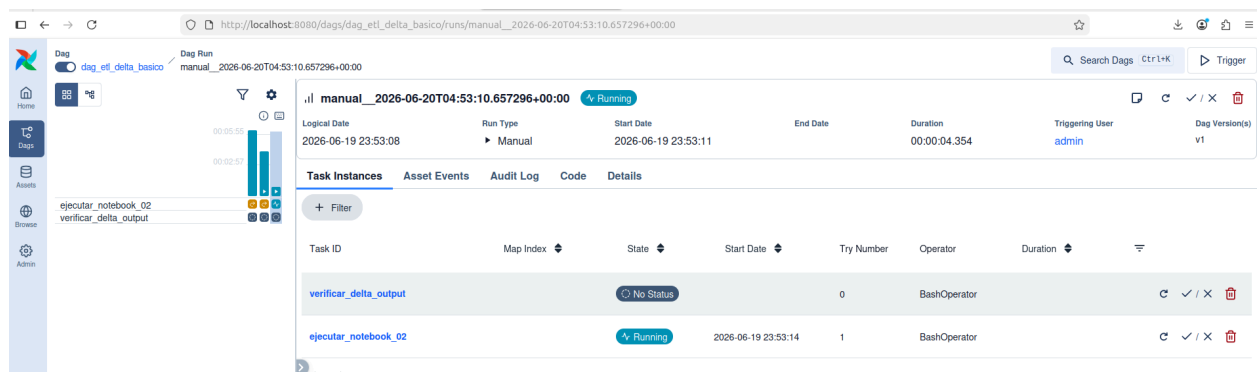
En la UI, localizar dag_etl_delta_basico

Activar (toggle a ON)

Clic en play → Trigger DAG

3. Monitorear la Ejecución

Clic en el nombre del DAG para ver la vista



Clic en cada tarea para ver:

Logs: Salida del comando ejecutado

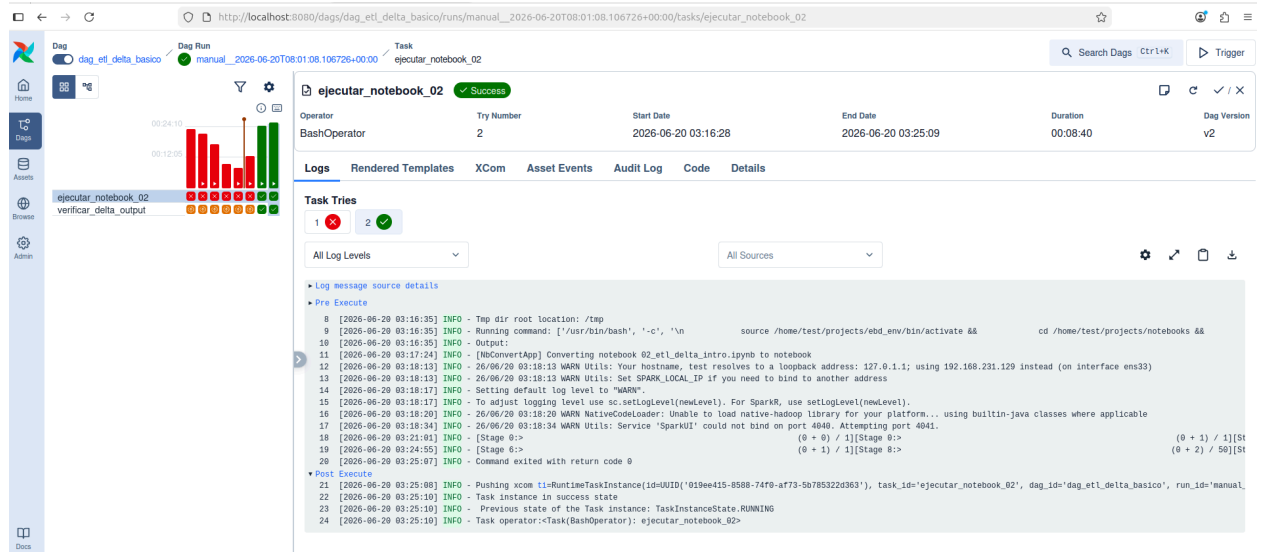
XCom: Datos transferidos entre tareas

4. Verificar Logs de la Tarea

Clic en la tarea ejecutar_notebook_02

Selecciona "Log"

Debe ver algo como:



5. Verificar que el Notebook se Ejecutó

Shell

```
ls -la ~/projects/notebooks/02_etl_delta_intro_executed.ipynb
```

Debe existir y tener fecha reciente